SIMATS ENGINEERING

CO2_AT1_Scenario-Based Assessment

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO2: *Design effective test cases using black-box, white-box testing techniques and to control methods flow.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	An online banking system is experiencing frequent failures during fund transfers, especially at boundary values (e.g., minimum and maximum transfer limits). As a test engineer, critically analyze the scenario and evaluate the current test case design strategy. Apply appropriate black box testing techniques such as equivalence partitioning and boundary value analysis to redesign the test cases. Assess the effectiveness of random testing used by the team and justify whether it should be retained or replaced. Propose a structured and optimized testing approach ensuring maximum defect detection. Present your solution with clear reasoning and justification.	5
2	A healthcare application module has undergone testing, but critical defects were found post-deployment due to insufficient coverage. Evaluate the testing approach used and analyze gaps in test adequacy criteria. Apply white box testing techniques by constructing a conceptual control flow graph and identifying missing paths. Critically assess the coverage achieved (statement, branch, or path coverage). Design an improved testing strategy ensuring complete coverage and justify your decisions. Explain how your approach enhances reliability and reduces risk.	5
3	A retail e-commerce system uses multiple input fields (price, quantity, discount), but testers failed to detect errors in specific input combinations. Analyze the scenario and evaluate the limitations of the current black box testing approach. Apply appropriate techniques such as equivalence class partitioning, boundary value analysis, and other black box methods (e.g., decision table testing). Propose a revised test design strategy covering all possible input combinations. Justify your approach in terms of improved defect detection and test completeness. Provide a structured explanation of your solution.	5

4	A software module has complex nested conditions and loops, but the testing team reports only partial path coverage. Evaluate the current white box testing approach and identify gaps in control flow graph construction and path testing. Analyze how inadequate identification of independent paths impacts defect detection. Propose a complete path testing strategy ensuring all critical paths are tested. Justify your decisions using coverage criteria and expected outcomes. Present your analysis in a clear and structured manner.	5
5	A financial application combines both black box and white box testing, yet defects continue to appear in production. Critically analyze the integration of test case design strategies used in the project. Evaluate whether test adequacy criteria and coverage metrics were properly applied. Propose an improved hybrid testing strategy combining black box techniques (ECP, BVA) and white box methods (path testing, CFG). Justify how your approach ensures better coverage and defect prevention. Conclude with a well-structured and reasoned solution.	5
6	A specialized HR payroll system calculates monthly bonuses based on years of service and performance ratings. Testers found that employees at exactly 5, 10, and 15 years were receiving incorrect payouts. Critically analyze the failure and evaluate the current test design. Apply Boundary Value Analysis (BVA) and Equivalence Class Partitioning (ECP) to redesign the test suite. Justify why these techniques are superior to exploratory testing for this specific financial logic.	5
7	An automated temperature control system for a laboratory uses complex if-else logic to manage heaters and coolers. Post-release, the system failed to trigger the alarm during a specific sensor-malfunction sequence. Construct a Control Flow Graph (CFG) for such a logic module and evaluate the gaps in branch coverage. Propose a white-box testing strategy that ensures 100% Modified Condition/Decision Coverage (MC/DC) and justify its impact on safety-critical reliability.	5
8	A travel booking portal allows users to apply multiple promo codes, but certain combinations lead to a negative "Total Amount" due. Analyze the limitations of testing these inputs in isolation. Apply Decision Table Testing or Cause-Effect Graphing to map all valid and invalid combinations of price, promo type, and user tier. Propose an optimized test set that maximizes defect detection while minimizing redundant test cases.	5

9	A legacy database migration tool uses a deeply nested loop structure to validate records. The team reports high statement coverage but frequent "infinite loop" crashes in production. Evaluate the current white-box approach and identify why statement coverage was insufficient. Propose a Path Testing strategy focusing on independent paths and loop boundary conditions. Justify your approach using Cyclomatic Complexity metrics.	5
10	A mobile food delivery app features a "Real-time Tracker" that transitions through various states (Ordered, Preparing, Out for Delivery, Delivered). Users reported the app getting stuck when a driver cancels mid-delivery. Evaluate the current black-box strategy and apply State Transition Testing to redesign the scenarios. Justify how this technique addresses the logic gaps that ECP and BVA might miss in dynamic workflows.	5
11	A cybersecurity authentication module uses a multi-factor logic involving password strength, OTP timing, and IP location. Despite testing, "Brute Force" vulnerabilities were detected. Critically assess the integration of black-box and white-box methods used. Propose a Hybrid Testing strategy that combines Error Guessing with formal Path Analysis. Justify how this integrated approach enhances the security posture of the module.	5
12	An e-learning platform provides tiered access (Free, Basic, Premium) based on subscription dates and course types. Errors were found when a user upgrades exactly on the day of expiration. Evaluate the adequacy of the current testing criteria. Apply Equivalence Class Partitioning to the date ranges and subscription types. Propose a structured test design and justify its completeness in ensuring seamless user transitions.	5
13	A smart-home lighting controller uses a specific algorithm to dim lights based on ambient light sensors. The code contains several switch-case statements and complex Boolean expressions. Evaluate the risks of relying solely on statement coverage for this module. Apply Branch and Condition Coverage techniques to identify missing logical paths. Design an improved white-box strategy and justify how it reduces the risk of hardware-software integration failures.	5
14	A logistics management system calculates shipping costs based on weight, destination zone, and delivery speed. Testers failed to detect errors when weights were entered as zero or negative values. Analyze the failure of the current input validation testing. Apply Boundary Value Analysis (Robustness Testing) to include invalid boundaries. Justify the necessity of testing "negative scenarios" for robust system performance.	5

15	A medical imaging software processes large datasets using parallel processing loops. Inconsistent results occur only when the data size is a prime number. Evaluate the testing team's reliance on random testing for such a specialized mathematical module. Propose a structured White-Box Testing approach that incorporates Basis Path Testing. Justify your solution by explaining how it ensures mathematical accuracy and system stability.	5

Solutions:

1. Online Banking System – Black Box Testing

The frequent failures in fund transfers indicate that the current test case design does not adequately cover boundary values and invalid inputs. Equivalence Class Partitioning (ECP) divides transfer amounts into valid and invalid groups, while Boundary Value Analysis (BVA) tests values near the minimum and maximum transfer limits. For example, if the permitted transfer amount is ₹100 to ₹50,000, test values such as ₹99, ₹100, ₹101, ₹49,999, ₹50,000, and ₹50,001. These cases help identify errors in validation conditions and incorrect handling of limits. Random testing alone is insufficient because it may not select important boundary values or specific error conditions.

The improved strategy should combine ECP, BVA, decision-based tests, and selected random tests. Test cases should cover valid transfers, insufficient balances, invalid account numbers, network failures, and duplicate transactions. Random testing can be retained as a supplementary technique for discovering unexpected defects, but it should not replace systematic testing. Each test case must have defined inputs, expected results, and pass/fail criteria. This structured approach improves defect detection, reduces redundant tests, and increases the reliability and security of online banking transactions.

2. Healthcare Application – White Box Testing

Critical defects discovered after deployment suggest that the healthcare application was tested without sufficient code coverage and risk analysis. White box testing examines the internal program structure, including statements, branches, conditions, and execution paths. A conceptual Control Flow Graph (CFG) for a healthcare module may contain nodes for receiving patient data, validating it, checking medical conditions, calculating results, and displaying alerts. The edges represent possible execution flows between these operations. Missing paths may include invalid patient data, abnormal test results, unavailable records, and error-handling conditions.

Statement coverage verifies that every statement executes at least once, while branch coverage checks both true and false outcomes of decisions. Path coverage examines different execution paths but can become impractical when loops and complex conditions produce many paths. The improved strategy should prioritize 100% statement and branch coverage, condition coverage for critical decisions, and selected path testing based on risk. Tests must also verify abnormal medical inputs and failure-handling procedures. This

approach improves reliability by identifying logical errors before deployment and reducing the possibility of incorrect medical results or unsafe decisions.

3. Retail E-Commerce System – Black Box Testing

The failure to detect errors in combinations of price, quantity, and discount indicates that testing each input independently was insufficient. Equivalence Class Partitioning divides inputs into valid and invalid categories, while Boundary Value Analysis tests values near limits. For example, price may be tested using zero, a valid positive value, and a negative value; quantity may be tested using zero, one, and its maximum permitted value; and discount may be tested using zero, the maximum allowed percentage, and a value above the limit. These tests identify validation errors and incorrect calculations near boundaries.

Decision Table Testing should also be used to examine combinations of conditions, such as valid price, available stock, valid quantity, and permitted discount. Test cases should verify the final amount, tax, discount, and rejection of invalid orders. Testing every possible combination may be expensive, so pairwise testing can cover interactions between input pairs, while higher-risk combinations receive additional tests. The revised strategy combines ECP, BVA, decision tables, and selected combination testing. This improves defect detection, reduces unnecessary repetition, and ensures that important input interactions are tested systematically.

4. Software Module – Path Testing

Partial path coverage in a module with nested conditions and loops indicates that the testing team may not have identified all important execution paths. A Control Flow Graph should be constructed with nodes representing statements or blocks and edges representing possible control transfers. Nested if-else conditions create alternative branches, while loops introduce repeated execution paths. If independent paths are not identified correctly, certain combinations of conditions, loop exits, and error-handling operations may remain untested, allowing logical defects to reach production.

The improved strategy should calculate Cyclomatic Complexity to estimate the number of linearly independent paths. For a connected control flow graph, it can be calculated as $V(G) = E - N + 2$, where E is the number of edges and N is the number of nodes. Each independent path should be tested at least once, along with loop execution for zero, one, and multiple iterations and relevant nested-condition combinations. Complete path coverage may be impossible when loops create infinitely many paths, so critical paths and loop boundaries should be prioritized. This strategy provides measurable coverage and improves confidence in the correctness of complex control logic.

5. Financial Application – Hybrid Testing

The continued appearance of production defects suggests that black box and white box testing may have been performed separately without sufficient integration. Black box testing checks externally visible behavior, whereas white box testing examines internal logic and execution paths. Equivalence Class Partitioning and Boundary Value Analysis can identify invalid financial inputs, incorrect transaction limits, and rounding errors. However, these techniques alone may miss defects caused by internal conditions,

incorrect branches, or untested combinations of operations. Similarly, statement coverage does not guarantee that all decision outcomes have been tested.

An improved hybrid strategy should combine ECP, BVA, decision tables, and negative testing with CFG analysis, branch coverage, condition coverage, and basis path testing. For example, payment amounts should be tested at their minimum and maximum values, while internal paths should be checked for successful payments, insufficient funds, transaction failures, and rollback operations. Coverage metrics should be reviewed alongside requirements traceability, defect severity, and risk-based priorities. Integration, regression, and security testing should also be included. This combined strategy improves overall coverage, identifies defects at multiple levels, and reduces the likelihood of financial errors in production.

6. HR Payroll System – Boundary Value Analysis

Incorrect bonus payments at exactly 5, 10, and 15 years indicate possible boundary-condition errors in the payroll algorithm. The program may use incorrect comparison operators, such as greater-than instead of greater-than-or-equal-to, or assign employees to the wrong service category. Exploratory testing may discover some of these defects, but it does not guarantee systematic testing of every important boundary. Equivalence Class Partitioning should divide employees into service groups, such as below 5 years, 5 to below 10 years, 10 to below 15 years, and 15 years or more, depending on the specified bonus rules.

Boundary Value Analysis should test values immediately below, exactly at, and immediately above each threshold. For example, test 4 years 11 months, exactly 5 years, and 5 years 1 month; repeat the same approach around 10 and 15 years. Performance ratings should also be divided into valid and invalid classes, and selected combinations of service duration and rating should be tested. The expected bonus must be calculated independently and compared with the system output. BVA and ECP are superior for this specific problem because they systematically target threshold errors and input categories, while exploratory testing can remain a supplementary technique for discovering unexpected behavior.

7. Laboratory Temperature Control – MC/DC Testing

The failure to trigger an alarm during a sensor-malfunction sequence suggests that important decision outcomes or condition combinations were not adequately tested. A conceptual Control Flow Graph should include sensor reading, sensor-validity checking, temperature comparison, heater or cooler activation, alarm evaluation, and program termination. The graph should represent both true and false outcomes of every decision, including abnormal sensor readings. Branch coverage alone may be insufficient because a decision can execute both outcomes without demonstrating that each individual condition independently affects the result.

The improved white box strategy should achieve 100% Modified Condition/Decision Coverage (MC/DC) for the applicable safety-critical decisions. MC/DC requires each atomic condition to be shown independently capable of changing the decision outcome while the other conditions are held appropriately constant. For example, tests should independently vary sensor validity, excessive temperature, and cooling-system status to verify when the alarm must activate. Each test should record sensor inputs, expected control actions, and actual outputs. Fault injection should simulate disconnected sensors, out-of-range readings, and malfunction sequences. This approach provides stronger evidence that

safety-related conditions work correctly and reduces the risk of undetected failures in the laboratory control system.

8. Travel Booking Portal – Decision Table Testing

Testing promo codes individually cannot reveal all errors caused by interactions between promotional offers, booking prices, and user subscription tiers. For example, applying two percentage discounts or combining a fixed discount with a promotional offer may reduce the total below zero. Equivalence Class Partitioning and Boundary Value Analysis can test valid and invalid prices, but Decision Table Testing is more suitable for verifying combinations of multiple conditions. The relevant conditions include whether the price is valid, whether each promo code is valid, whether the codes can be combined, and whether the user tier is eligible for a discount.

The decision table should define the expected outcome for valid combinations, ineligible users, expired codes, incompatible promotions, and discounts exceeding the booking price. Test cases should verify that the final amount never becomes negative and that invalid combinations are rejected or handled according to the business rules. Redundant combinations can be reduced by grouping equivalent conditions and using pairwise testing for lower-risk interactions, while high-risk combinations receive dedicated tests. This structured approach improves coverage of interacting inputs without requiring every possible combination. It also ensures that discount calculations follow business rules and that incorrect totals are detected before deployment.

9. Legacy Database Migration – Path Testing

High statement coverage combined with infinite-loop crashes indicates that executing every statement once was not enough to verify loop termination. Statement coverage does not guarantee that loops execute zero times, multiple times, or terminate correctly under different conditions. A deeply nested database-validation process may contain loops for reading records, validating fields, retrying failed operations, and processing related records. If loop conditions are incorrectly updated or exit conditions are missing, the program may repeatedly process the same records even when statement coverage is high.

The improved strategy should construct a Control Flow Graph, calculate Cyclomatic Complexity, and identify independent execution paths using basis path testing. Cyclomatic Complexity can be calculated as $V(G) = E - N + 2$ for a connected graph. Test cases should cover zero, one, and multiple loop iterations, empty datasets, single-record datasets, large datasets, invalid records, and termination conditions. Nested loops should also be tested with different inner and outer iteration counts. Because exhaustive path testing may be impossible, the team should prioritize independent paths and loop boundaries, using timeouts and iteration limits to detect non-termination. This strategy provides better evidence of loop correctness and reduces infinite-loop failures during migration.

10. Food Delivery App – State Transition Testing

The delivery tracker becoming stuck when a driver cancels indicates that the current black box testing did not sufficiently cover state changes and exceptional events. Equivalence Class Partitioning and Boundary Value Analysis are useful for input values, but they cannot fully verify dynamic workflows involving

ordered sequences of events. State Transition Testing models the application states as Ordered, Preparing, Out for Delivery, Delivered, and Cancelled. It also defines valid transitions, such as Ordered to Preparing and Out for Delivery to Delivered, along with exceptional transitions caused by driver cancellation, customer cancellation, or delivery failure.

The revised strategy should test every valid transition, important invalid transitions, and relevant event sequences. For example, test cancellation before preparation, cancellation during delivery, repeated cancellation requests, and driver reassignment after cancellation. The expected state, user notification, order status, and refund behavior should be verified after each event. Transition coverage and selected transition-pair coverage should be measured to identify missing scenarios. This technique is particularly effective because it tests not only individual inputs but also the order in which events occur. It helps prevent stuck states, inconsistent order statuses, and incorrect notifications in the delivery application.

11. Cybersecurity Authentication – Hybrid Testing

The discovery of brute-force vulnerabilities indicates that the existing testing strategy may not have adequately tested repeated authentication failures, rate limits, or interactions between authentication conditions. Black box testing can verify externally visible behavior, including password validation, OTP expiration, account lockout, and IP-based restrictions. However, it may miss internal control-flow weaknesses, such as incorrect retry-counter updates or branches that bypass security checks. Error Guessing is useful for identifying likely weaknesses, including repeated OTP submissions, expired OTP reuse, rapid login attempts, and attempts from unfamiliar locations.

The improved hybrid strategy should combine Error Guessing and negative testing with Control Flow Graph analysis and formal path testing. The CFG should include password verification, OTP generation and validation, IP risk assessment, retry counting, lockout, and successful authentication. Tests should verify that every failure path enforces the appropriate security controls and that no path bypasses multifactor authentication. Rate limiting, session management, audit logging, and account recovery should also be tested. Security testing should include controlled brute-force simulations and repeated-request tests under authorized conditions. This approach improves the detection of logical weaknesses and helps ensure that authentication controls remain effective under malicious or abnormal usage.

12. E-Learning Subscription – Equivalence Class Partitioning

Errors during upgrades on the exact expiration date suggest that the current test cases did not adequately cover date boundaries and subscription transitions. Equivalence Class Partitioning should divide subscription dates into active, expired, and future-dated categories. Subscription types should also be divided into Free, Basic, and Premium classes, with additional categories for valid upgrades, downgrades, and invalid transitions. The application must define whether access remains valid throughout the expiration date or ends at a specific time, including the applicable time zone. Without a clear rule, the same date may be interpreted differently by the application and the user.

The revised test suite should test the day before expiration, the exact expiration date, and the day after expiration. It should also cover upgrades before expiration, upgrades on expiration, upgrades after expiration, expired payment methods, and successful or failed payment processing. Each test should verify

the subscription status, course permissions, billing amount, and effective upgrade date. Boundary Value Analysis complements ECP by checking the exact time and date limits, while decision tables can cover subscription-type combinations. This approach systematically identifies transition errors and ensures that users receive the correct access rights without unintended billing or service interruptions.

13. Smart-Home Lighting Controller – Branch and Condition Coverage

Relying only on statement coverage is risky because a lighting controller may execute every statement without testing both outcomes of its decisions. Complex Boolean expressions and switch-case statements can contain defects that appear only under specific combinations of ambient light readings, manual settings, motion detection, and operating modes. For example, a statement that dims the lights may execute during testing, while the branch that restores brightness under low ambient light remains untested. Missing switch cases and incorrect logical operators can also cause unexpected hardware behavior.

The improved white box strategy should construct a Control Flow Graph and apply statement, branch, and condition coverage. Every switch case, default case, and true or false outcome should be tested. For compound Boolean expressions, condition coverage should verify that each individual condition takes both true and false values; MC/DC may be added for critical decisions. Tests should include sensor readings at operational thresholds, disconnected sensors, conflicting manual and automatic settings, and unexpected input values. Hardware-in-the-loop testing should verify that the software commands produce the intended physical light response. This strategy provides stronger logical coverage and reduces the risk of software errors causing incorrect hardware operation.

14. Logistics Management – Robust Boundary Value Analysis

The failure to detect zero and negative shipping weights indicates that input validation testing did not include invalid boundary values. If the system accepts a weight of zero or a negative number, it may calculate an incorrect shipping charge or allow an invalid shipment to proceed. Boundary Value Analysis should test values immediately below, at, and above the minimum permitted weight. For example, if the minimum weight is greater than zero, test a negative weight, zero, the smallest permitted positive weight, and a value just above that minimum. The maximum permitted weight and values exceeding it should also be tested.

Robustness testing extends ordinary boundary testing by including invalid values outside the permitted range. The revised test suite should combine these tests with Equivalence Class Partitioning for destination zones, delivery speeds, and valid or invalid weights. Expected results should include correct shipping charges for valid inputs and clear validation errors for invalid inputs. Additional tests should cover missing weights, non-numeric values, and excessively large numbers. Negative scenarios are essential because real users and external systems may supply incorrect data. This approach prevents invalid calculations, strengthens input validation, and improves the overall reliability of the logistics application.

15. Medical Imaging Software – Basis Path Testing

Inconsistent results for datasets whose sizes are prime numbers suggest that random testing has not adequately explored the mathematical and control-flow conditions involved in parallel processing.

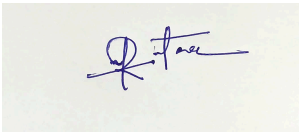
Randomly selected dataset sizes may fail to expose errors related to partitioning, thread allocation, uneven workload distribution, or remainder handling. Statement execution alone also cannot guarantee that the algorithm processes every data element correctly. The testing team should examine the internal logic responsible for dividing datasets, distributing tasks among threads, combining partial results, and handling incomplete partitions.

The improved white box strategy should construct a Control Flow Graph and apply Basis Path Testing to identify independent execution paths. Cyclomatic Complexity can estimate the number of independent paths that should be considered. Tests should include empty, small, large, even-sized, odd-sized, and prime-sized datasets, along with sizes that are not evenly divisible by the number of processing threads. Results should be compared with a trusted sequential implementation to verify mathematical accuracy. Boundary tests, thread-scheduling variations, repeated executions, and stress tests should also be performed to detect race conditions and resource failures. This structured strategy complements random testing and improves confidence in both computational correctness and system stability.

Code of Conduct:

I Prartana T(192421099) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding & Context Analysis	25	Thoroughly understands the scenario with accurate interpretation of all requirements	Clearly understands the scenario with minor gaps	Basic understanding; some aspects of the scenario unclear	Poor understanding or misinterpretation of the scenario
Application of Concepts	30	Applies appropriate concepts effectively to solve complex real-world problems	Applies relevant concepts correctly with minor errors	Applies basic concepts but with limited accuracy	Fails to apply appropriate concepts
Analytical & Decision-Making Skills	20	Demonstrates strong analysis and selects optimal solutions with justification	Good analysis with reasonable solutions	Limited analysis; solutions lack justification	Poor or no analysis; incorrect decisions
Solution Design & Approach	15	Well-structured, innovative, and feasible solution approach	Logical and structured solution with minor gaps	Basic solution with limited structure or feasibility	Unclear or incorrect solution approach
Clarity, Justification & Presentation	10	Clear, well-justified explanation with strong reasoning	Clear explanation with some justification	Limited clarity and weak justification	Poorly explained with no justification